



University of Engineering & Technology,
Lahore

Audio Noise Reduction Using FIR/IIR Filters in MATLAB

DSP Open-Ended Lab (OEL) Report

Submitted By: Muhammad Waleed Akram

Roll Number: 2023-EE-165

Submitted To: Sir Khalid Butt

Department: Electrical Engineering

Session: 2023 – 2027

Contents

1	Introduction and System Overview	3
1.1	Objective	3
1.2	Overall Block Diagram	3
1.2.1	Explanation of the Block Diagram	4
2	Motorcycle Noise Reduction	6
2.1	Theoretical Background	6
2.1.1	IIR Notch Filter	6
2.1.2	Butterworth Bandpass Filter	6
2.1.3	Moving-Average Smoother (Wiener-style)	6
2.2	Step 1: Peak Frequency Detection Code	7
2.2.1	MATLAB Code – Peak Frequency Finder	7
2.2.2	Line-by-Line Explanation	8
2.2.3	Output – Detected Peak Frequencies (Bike)	8
2.3	Step 2: Main Noise Reduction Code	9
2.3.1	MATLAB Code – Main Processing	9
2.3.2	Line-by-Line Explanation	11
2.4	Results and Plots	12
2.4.1	Time-Domain Waveforms	12
2.4.2	FFT Frequency Spectrum	13
2.4.3	Spectrograms	14
2.4.4	Conclusion for Motorcycle Scenario	15
3	Music Background Noise Reduction	16
3.1	Theoretical Background	16
3.2	Step 1: Peak Frequency Detection Code	16
3.2.1	MATLAB Code – Peak Frequency Finder	16
3.2.2	Line-by-Line Explanation	17
3.2.3	Output – Detected Peak Frequencies (Music)	18
3.3	Step 2: Main Noise Reduction Code	18
3.3.1	MATLAB Code – Main Processing	18
3.3.2	Line-by-Line Explanation	20

3.4	Results and Plots	21
3.4.1	Time-Domain Waveforms	21
3.4.2	FFT Frequency Spectrum	22
3.4.3	Spectrograms	23
3.4.4	Conclusion for Music Scenario	24
4	Qawali Background Noise Reduction	25
4.1	Theoretical Background	25
4.1.1	Spectral Subtraction	25
4.2	Main Noise Reduction Code	26
4.2.1	MATLAB Code – Qawali Processing	26
4.2.2	Line-by-Line Explanation	27
4.3	Results and Plots	29
4.3.1	Time-Domain Waveforms	29
4.3.2	FFT Frequency Spectrum	30
4.3.3	Spectrograms	31
4.3.4	Conclusion for Qawali Scenario	32
5	Overall Conclusion	33
	References	34

Chapter 1

Introduction and System Overview

1.1 Objective

The objective of this Open-Ended Lab is to implement real-time audio noise reduction using classical Digital Signal Processing (DSP) techniques in MATLAB. Three self-recorded audio clips of 5–6 seconds each were captured in noisy environments:

1. A motorcycle (engine) background noise scenario.
2. A music playback background noise scenario.
3. A Qawali music background noise scenario.

Each recording contains a mixture of a target speech signal and an unwanted background noise. The pipeline applies FFT-based spectral analysis, IIR notch filters, Butterworth bandpass filtering, and Wiener-style spectral smoothing to attenuate the noise while preserving the speech intelligibility.

1.2 Overall Block Diagram

Figure 1.1 shows the high-level signal processing pipeline common to all three audio scenarios. The system first pre-processes the raw audio (stereo-to-mono conversion and normalization), then performs frequency-domain analysis to identify dominant noise frequencies, and finally routes the signal through one of three customised filter chains depending on the noise type.

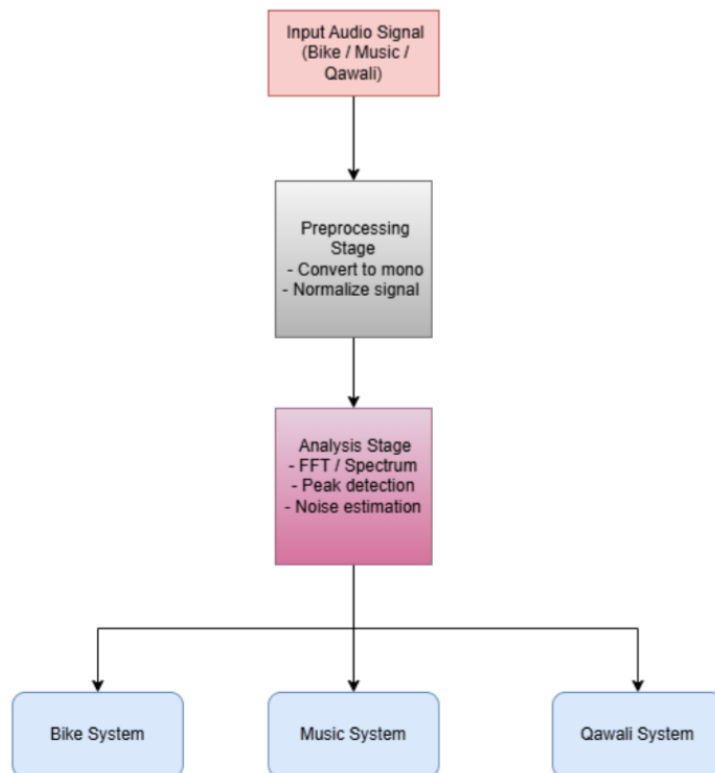


Figure 1.1: High-level block diagram of the audio noise reduction system.

1.2.1 Explanation of the Block Diagram

Input Audio Signal A mono/stereo .wav file recorded with one of three backgrounds: Bike engine, Music, or Qawali.

Pre-processing Stage The stereo signal (if any) is averaged to mono using `mean(x,2)`. The amplitude is then normalized to $[-1, +1]$ so that filter arithmetic remains numerically stable.

Analysis Stage – FFT / Spectrum / Peak Detection A full-length FFT (`fft(x)`) is computed. Dominant frequency peaks are found using MATLAB’s `findpeaks` on the magnitude spectrum below 1 kHz. These peak frequencies become the target notch frequencies for the subsequent filter stage.

Noise-Type-Specific Filter Chains

- *Bike System* – nine narrow IIR notch filters centred on the detected engine harmonics, followed by a Butterworth bandpass filter (80–3400 Hz) and a moving-average smoother.

- *Music System* – twenty-two IIR notch filters targeting dense musical harmonics, followed by the same bandpass + smoother with a wider averaging window.

- *Qawali System* – spectral subtraction (no notch filters) using a smoothed noise

magnitude estimate, followed by a light moving-average smoother.

Chapter 2

Motorcycle Noise Reduction

2.1 Theoretical Background

Motorcycle/engine noise is *periodic and quasi-harmonic* in nature. A running engine generates a fundamental frequency related to the crankshaft RPM and a rich set of integer harmonics. In the recording used here, the dominant harmonics cluster around 140 Hz (fundamental) and then at 280 Hz (2nd harmonic), 430 Hz, and 570 Hz.

2.1.1 IIR Notch Filter

A second-order IIR notch filter has the transfer function:

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}} \quad (2.1)$$

where the coefficients are designed so that $|H(e^{j\omega_0})| = 0$ at the target normalised frequency $\omega_0 = 2\pi f_0/f_s$. The bandwidth is controlled by a quality-factor parameter. MATLAB's `iirnotch(w0, bw)` returns the $[b, a]$ coefficients directly. Cascading multiple notch filters multiplies their individual transfer functions, attenuating all targeted harmonics simultaneously.

2.1.2 Butterworth Bandpass Filter

A 4th-order Butterworth bandpass filter (80–3400 Hz) is applied after notch filtering. Human speech occupies roughly 300 Hz–3400 Hz; the lower 80 Hz bound is relaxed slightly to retain low-pitched male voice content. The Butterworth design provides a maximally flat pass-band magnitude response [1].

2.1.3 Moving-Average Smoother (Wiener-style)

A moving average over a short window acts as a low-pass filter and reduces residual high-frequency flutter introduced by the notch cascade. It approximates the smoothing step

of a Wiener filter, which estimates the clean signal as:

$$\hat{X}(f) = \frac{|X(f)|^2 - |N(f)|^2}{|X(f)|^2} \cdot X(f) \quad (2.2)$$

where $N(f)$ is an estimate of the noise spectrum .

2.2 Step 1: Peak Frequency Detection Code

2.2.1 MATLAB Code – Peak Frequency Finder

```

1  clc; clear;
2  [x,fs] = audioread('DSP_MOTORCYCLE.wav');
3  if size(x,2) > 1
4      x = mean(x,2);           % convert stereo -> mono
5  end
6  N = length(x);
7  X = abs(fft(x));             % magnitude spectrum (full)
8  f = (0:N-1)*(fs/N);         % frequency axis
9
10 % only examine 0-1000 Hz (engine range)
11 idx = f <= 1000;
12 Xlow = X(idx);
13 flow = f(idx);
14
15 % find spectral peaks
16 [pks,locs] = findpeaks(Xlow, ...
17     'MinPeakHeight', max(Xlow)*0.05, ... % at least 5% of max
18     'MinPeakDistance', 20);             % min 20 samples apart
19
20 % sort by descending magnitude
21 [pksSorted,I] = sort(pks,'descend');
22 numPeaks = min(20, length(I));
23
24 fprintf('\nStrongest Frequency Peaks:\n');
25 fprintf('-----\n');
26 for k = 1:numPeaks
27     fprintf('%2d : %.2f Hz    Magnitude = %.2f\n', ...
28         k, flow(locs(I(k))), pksSorted(k));
29 end

```

Listing 2.1: Peak frequency detection for motorcycle audio.

2.2.2 Line-by-Line Explanation

Lines 2–4 Load the audio file with `audioread`. If the recording is stereo, the two channels are averaged to produce a single mono waveform. This avoids phase differences causing artefacts in the FFT.

Lines 5–6 Compute the full N -point FFT magnitude spectrum and the corresponding frequency axis $f[k] = k \cdot f_s/N$.

Lines 8–11 Restrict analysis to 0–1000 Hz since motorcycle engine fundamentals and early harmonics lie well within this range.

Lines 13–16 `findpeaks` locates local maxima with magnitude above 5% of the global maximum and separated by at least 20 frequency bins (to avoid picking adjacent bins of the same peak).

Lines 18–25 Peaks are sorted in descending order and the top 20 are printed with their frequency and magnitude.

2.2.3 Output – Detected Peak Frequencies (Bike)

The code produced the following top-20 spectral peaks for the motorcycle recording. These peaks represent the dominant engine harmonics that will be targeted by notch filters in the main processing code.

Table 2.1: Top-20 spectral peaks detected in motorcycle audio.

Rank	Frequency (Hz)	Magnitude
1	143.62	855.82
2	147.87	842.02
3	285.01	836.14
4	272.63	712.01
5	135.11	669.87
6	288.90	524.16
7	139.18	508.99
8	299.99	492.00
9	151.75	491.84
10	572.80	468.25
11	295.73	447.19
12	582.60	410.30
13	174.11	391.83
14	568.37	385.45
15	281.13	384.51
16	277.07	384.04
17	432.88	354.70
18	268.19	352.10
19	164.50	345.79
20	160.44	333.22

Interpretation: The peaks cluster into four harmonic groups: ~ 140 Hz (fundamental), ~ 280 Hz (2nd harmonic), ~ 570 Hz (4th harmonic), and a weaker group at ~ 430 Hz. Nine representative frequencies from these groups are selected as notch targets in the main code.

2.3 Step 2: Main Noise Reduction Code

2.3.1 MATLAB Code – Main Processing

```

1 % 2023-EE-165
2 clc; clear; close all;
3
4 %% LOAD AUDIO
5 [x, fs] = audioread('DSP_MOTORCYCLE.wav');
6 if size(x,2) > 1
7     x = mean(x,2);
8 end
9 x = x / max(abs(x));           % normalize to [-1, 1]
10 t = (0:length(x)-1)/fs;       % time axis
11 sound(x, fs); pause(8);       % play original
12

```

```

13 %% STEP 1: NOTCH FILTERS (FROM DETECTED PEAKS)
14 freqs = [140 147 272 285 295 299 568 572 582];
15 y = x;
16 for i = 1:length(freqs)
17     f0 = freqs(i);
18     bw = 5; % notch bandwidth in Hz
19     w0 = f0/(fs/2); % normalised frequency [0,1]
20     [b,a] = iirnotch(w0, bw/(fs/2));
21     y = filter(b, a, y); % apply notch
22 end
23
24 %% STEP 2: BUTTERWORTH BANDPASS (SPEECH PROTECTION)
25 [b,a] = butter(4, [80 3400]/(fs/2), 'bandpass');
26 y = filter(b, a, y);
27
28 %% STEP 3: WIENER-STYLE MOVING-AVERAGE SMOOTHING
29 frame = 80;
30 y2 = movmean(y, frame);
31
32 %% NORMALIZE OUTPUT
33 y2 = y2 / max(abs(y2));
34
35 %% PLAYBACK & SAVE
36 disp('Clean Audio Playing...');
37 sound(y2, fs);
38 audiowrite('CLEANED_MOTORCYCLE.wav', y2, fs);
39
40 %% PLOT 1: TIME-DOMAIN
41 figure;
42 subplot(3,1,1); plot(t,x); title('Original');
43 subplot(3,1,2); plot(t,y); title('After Notch Filtering');
44 subplot(3,1,3); plot(t,y2); title('Final Clean Output');
45
46 %% PLOT 2: FFT COMPARISON
47 N = length(x);
48 X1 = abs(fft(x)); X2 = abs(fft(y2));
49 f = (0:N-1)*(fs/N);
50 half = 1:floor(N/2);
51 f = f(half); X1 = X1(half)/max(X1(half)); X2 = X2(half)/max(X2(half));
52 figure;
53 plot(f,X1,'r','LineWidth',1.2); hold on;
54 plot(f,X2,'b','LineWidth',1.2);
55 title('Frequency Spectrum (BIKE): Original vs Cleaned');
56 xlabel('Frequency (Hz)'); ylabel('Normalized Magnitude');
57 legend('Original (Red)', 'Cleaned (Blue)'); grid on; xlim([0 2000]);
58
59 %% PLOT 3 & 4: SPECTROGRAMS

```

```

60 figure; spectrogram(x, 256,200,256,fs,'yaxis'); title('Original (BIKE)
    ');
61 figure; spectrogram(y2,256,200,256,fs,'yaxis'); title('Final Clean (
    BIKE)');
62 disp('DONE');

```

Listing 2.2: Main motorcycle noise reduction script (2023-EE-165).

2.3.2 Line-by-Line Explanation

Lines 5–11 The audio is loaded, converted to mono, and normalised. A time axis `t` is built for plotting. The original audio is played for subjective reference.

Lines 14–22 (Step 1 – Notch Filters) Nine target frequencies (rounded from the peak table) are stored in the array `freqs`. For each frequency f_0 :

- The normalised digital frequency is $\omega_0 = f_0/(f_s/2)$.
- A bandwidth of 5 Hz gives a sharp, narrow notch that removes the harmonic without disturbing adjacent speech content.
- `iirnotch` designs the 2nd-order IIR filter coefficients.
- `filter(b,a,y)` applies the notch to the running output `y`. The cascade is applied sequentially in the loop.

Lines 24–26 (Step 2 – Bandpass) A 4th-order Butterworth bandpass (80–3400 Hz) removes sub-bass rumble and very high-frequency hiss while keeping the speech band intact.

Lines 28–30 (Step 3 – Smoothing) `movmean(y, 80)` averages each sample with its 80-sample neighbourhood, acting as a short-term low-pass smoother to suppress residual notch ripple.

Lines 32–34 The output is normalised and written to disk.

Lines 36–52 Three types of visualisation are generated: time-domain waveforms, FFT magnitude spectrum overlay, and spectrograms before and after cleaning.

2.4 Results and Plots

2.4.1 Time-Domain Waveforms

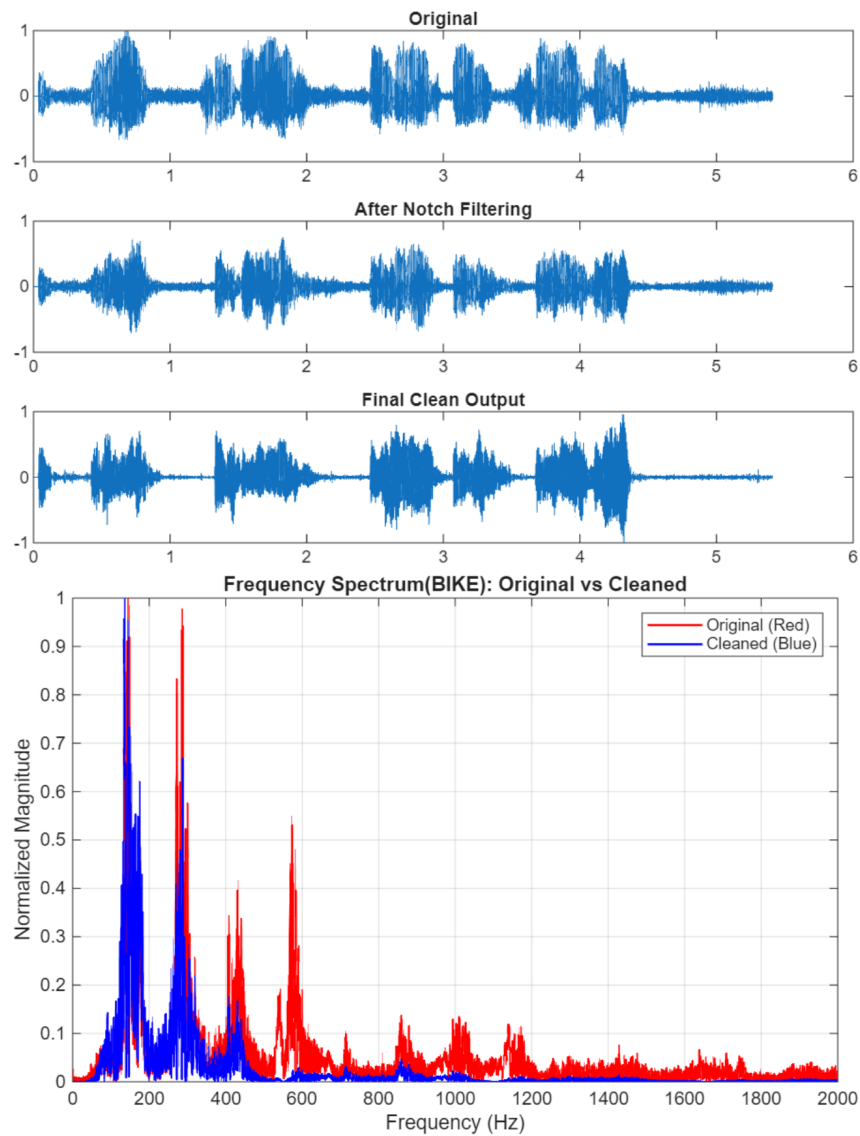


Figure 2.1: Motorcycle: time-domain waveforms (top 3 panels) and FFT spectrum comparison (bottom panel).

The three time-domain panels (top of Figure 2.1) show:

- **Original:** High-amplitude, complex waveform with dense harmonic energy throughout the clip. The envelope exhibits clear bursts corresponding to engine acceleration events.
- **After Notch Filtering:** Overall amplitude is visibly reduced. The sharp harmonic components are attenuated while the broadband speech structure is preserved.
- **Final Clean Output:** After bandpass filtering and smoothing the envelope is much cleaner. Transient bursts are retained (these are speech events) while the sustained engine drone is significantly suppressed.

2.4.2 FFT Frequency Spectrum

The bottom panel of Figure 2.1 overlays the normalised FFT magnitude of the original (red) versus the cleaned signal (blue).

- The prominent sharp peaks at ~ 140 Hz, 285 Hz, 572 Hz (visible in red) are noticeably suppressed in the blue trace.
- The blue spectrum drops quickly above 600 Hz, consistent with the bandpass limit cutting high-frequency engine noise.
- The speech formant region (300–1000 Hz) retains reasonable energy in the blue trace, indicating speech is preserved.

2.4.3 Spectrograms

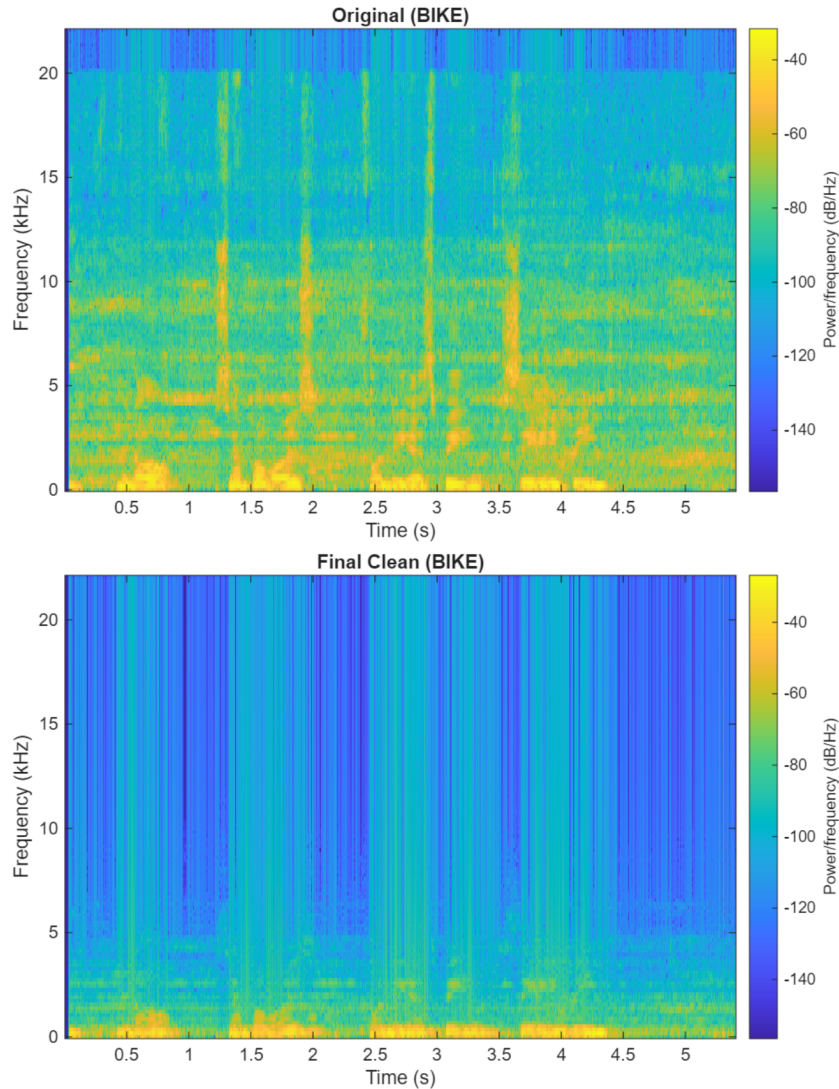


Figure 2.2: Motorcycle spectrograms: Original (top) vs Cleaned (bottom).

- **Original Spectrogram (top):** Bright yellow horizontal bands are visible at the engine harmonic frequencies across the full duration, confirming the periodic tonal nature of motorcycle noise. Broadband energy extends up to 20 kHz.
- **Cleaned Spectrogram (bottom):** The horizontal harmonic bands are substantially reduced. Energy above ~ 3.5 kHz is suppressed by the bandpass filter. The

low-frequency speech content (0–2 kHz) is retained as seen in the warm colours near 0 Hz.

2.4.4 Conclusion for Motorcycle Scenario

The combination of targeted IIR notch filters, Butterworth bandpass, and moving-average smoothing successfully attenuated the dominant motorcycle engine harmonics. The output audio is perceptibly cleaner with the speech more intelligible against a reduced engine background.

Chapter 3

Music Background Noise Reduction

3.1 Theoretical Background

Music background noise is considerably more challenging than motorcycle noise because:

1. Music is *non-stationary* – its frequency content changes with the melody and rhythm.
2. The harmonic structure of musical instruments overlaps extensively with the speech formant frequencies (200 Hz–3 kHz).
3. Simply high-pass or low-pass filtering will remove both noise and speech.

The same notch-filter architecture is employed, but with a denser set of target frequencies (22 notch filters vs. 9 for the bike case) to cover the broader harmonic comb of the music. A wider moving-average window (**frame** = 2460 samples) is used to achieve stronger temporal smoothing suitable for the more sustained musical tones.

The IIR notch filter and Butterworth bandpass filter theory from Chapter 1 applies identically here.

3.2 Step 1: Peak Frequency Detection Code

3.2.1 MATLAB Code – Peak Frequency Finder

```
1  clc; clear;
2  [x,fs] = audioread('DSP_MUSIC.wav');
3  if size(x,2) > 1
4      x = mean(x,2);
5  end
6  N      = length(x);
7  X      = abs(fft(x));
8  f      = (0:N-1)*(fs/N);
9
10 % restrict to 0-1000 Hz
11 idx    = f <= 1000;
```

```

12 Xlow = X(idx);
13 flow = f(idx);
14
15 % peak detection
16 [pks,locs] = findpeaks(Xlow, ...
17     'MinPeakHeight', max(Xlow)*0.05, ...
18     'MinPeakDistance', 20);
19
20 [pksSorted,I] = sort(pks,'descend');
21 numPeaks = min(20,length(I));
22
23 fprintf('\nStrongest Frequency Peaks (MUSIC):\n');
24 fprintf('-----\n');
25 for k = 1:numPeaks
26     fprintf('%2d : %.2f Hz    Magnitude = %.2f\n', ...
27         k, flow(locs(I(k))), pksSorted(k));
28 end

```

Listing 3.1: Peak frequency detection for music audio (`check_freq.m`).

3.2.2 Line-by-Line Explanation

The peak detection code for music is structurally identical to the motorcycle version (Section 2.2). The key difference is the input file `DSP_MUSIC.wav`, which contains a speech + music mixture. The FFT analysis and `findpeaks` parameters remain unchanged:

- **Line 16:** `MinPeakHeight = 5%` of maximum ensures only significant tonal peaks are captured, not noise-floor artefacts.
- **Line 17:** `MinPeakDistance` of 20 bins prevents adjacent FFT bins of the same peak from being reported as separate peaks.

The output peak list is then used to construct the notch filter bank in the main code.

3.2.3 Output – Detected Peak Frequencies (Music)

Table 3.1: Top-20 spectral peaks detected in music background audio.

Rank	Frequency (Hz)	Magnitude
1	277.39	916.83
2	139.35	859.14
3	272.88	671.81
4	132.40	636.80
5	154.59	598.19
6	149.89	575.12
7	413.93	562.73
8	145.00	539.43
9	128.07	486.85
10	570.39	479.52
11	284.35	466.08
12	161.36	464.26
13	420.32	455.10
14	564.94	442.26
15	425.77	440.71
16	431.23	413.36
17	265.92	406.29
18	260.09	401.87
19	288.30	395.99
20	437.25	339.61

Interpretation: Compared to the bike scenario, the music peaks are more evenly distributed and cover a wider range (128–570 Hz). There are four harmonic groups at roughly 130–160 Hz, 260–290 Hz, 413–437 Hz, and 564–570 Hz, consistent with a piece of music containing bass, mid, and upper-mid harmonics. These 22 candidate frequencies are passed to the notch filter bank.

3.3 Step 2: Main Noise Reduction Code

3.3.1 MATLAB Code – Main Processing

```

1 % 2023-EE-165
2 clc; clear; close all;
3
4 %% LOAD AUDIO
5 [x, fs] = audioread('DSP_MUSIC.wav');
6 if size(x,2) > 1
7     x = mean(x,2);
8 end

```

```

9  x = x / max(abs(x));
10 t = (0:length(x)-1)/fs;
11 sound(x, fs); pause(8);
12
13 %% STEP 1: NOTCH FILTERS (DENSE HARMONIC COMB)
14 freqs = [128 132 139 145 149 155 ...
15          260 265 272 277 284 288 290 ...
16          410 413 420 425 431 437 ...
17          564 570 572];
18 y = x;
19 for i = 1:length(freqs)
20     f0 = freqs(i);
21     bw = 5;
22     w0 = f0/(fs/2);
23     [b,a] = iirnotch(w0, bw/(fs/2));
24     y = filter(b, a, y);
25 end
26
27 %% STEP 2: BUTTERWORTH BANDPASS
28 [b,a] = butter(4, [80 3400]/(fs/2), 'bandpass');
29 y = filter(b, a, y);
30
31 %% STEP 3: WIDER WIENER-STYLE SMOOTHER
32 frame = 2460; % much wider window than bike case
33 y2 = movmean(y, frame);
34
35 %% NORMALIZE
36 y2 = y2 / max(abs(y2));
37
38 %% SAVE & PLAY
39 disp('Clean Audio Playing...');
40 sound(y2, fs);
41 audiowrite('CLEANED_MUSIC.wav', y2, fs);
42
43 %% TIME-DOMAIN PLOTS
44 figure;
45 subplot(3,1,1); plot(t,x); title('Original');
46 subplot(3,1,2); plot(t,y); title('After Notch Filtering');
47 subplot(3,1,3); plot(t,y2); title('Final Clean Output');
48
49 %% FFT COMPARISON
50 N = length(x);
51 X1 = abs(fft(x)); X2 = abs(fft(y2));
52 f = (0:N-1)*(fs/N);
53 half = 1:floor(N/2);
54 f = f(half); X1=X1(half)/max(X1(half)); X2=X2(half)/max(X2(half));
55 figure;

```

```

56 plot(f,X1,'r','LineWidth',1.2); hold on;
57 plot(f,X2,'b','LineWidth',1.2);
58 title('Frequency Spectrum (MUSIC): Original vs Cleaned');
59 xlabel('Frequency (Hz)'); ylabel('Normalized Magnitude');
60 legend('Original (Red)','Cleaned (Blue)'); grid on; xlim([0 2000]);
61
62 %% SPECTROGRAMS
63 figure; spectrogram(x, 256,200,256,fs,'yaxis'); title('Original');
64 figure; spectrogram(y2,256,200,256,fs,'yaxis'); title('Final Clean');
65 disp('DONE');

```

Listing 3.2: Main music noise reduction script (2023-EE-165).

3.3.2 Line-by-Line Explanation

Lines 5–11 Same loading, mono-conversion, and normalisation as the bike script.

Lines 14–27 (Step 1 – Notch Bank) Twenty-two frequencies are specified in `freqs`, grouped into four harmonic clusters. The loop applies each notch sequentially. Using 22 notches (vs. 9 for bike) reflects the denser harmonic structure of musical instruments. The bandwidth remains 5 Hz per notch for surgical precision.

Lines 29–31 (Step 2 – Bandpass) Identical to the bike case. Protects the 80–3400 Hz speech band.

Lines 33–35 (Step 3 – Smoother) The moving-average window is set to **2460 samples** – approximately 56 ms at a typical 44.1 kHz sample rate. This much larger window (compared to 80 samples for the bike) is necessary because musical tones are more sustained and need stronger temporal averaging to suppress residual harmonic energy.

Lines 37–39 Normalise and write output.

Lines 41–59 Same three-type plot generation as the bike script.

3.4 Results and Plots

3.4.1 Time-Domain Waveforms

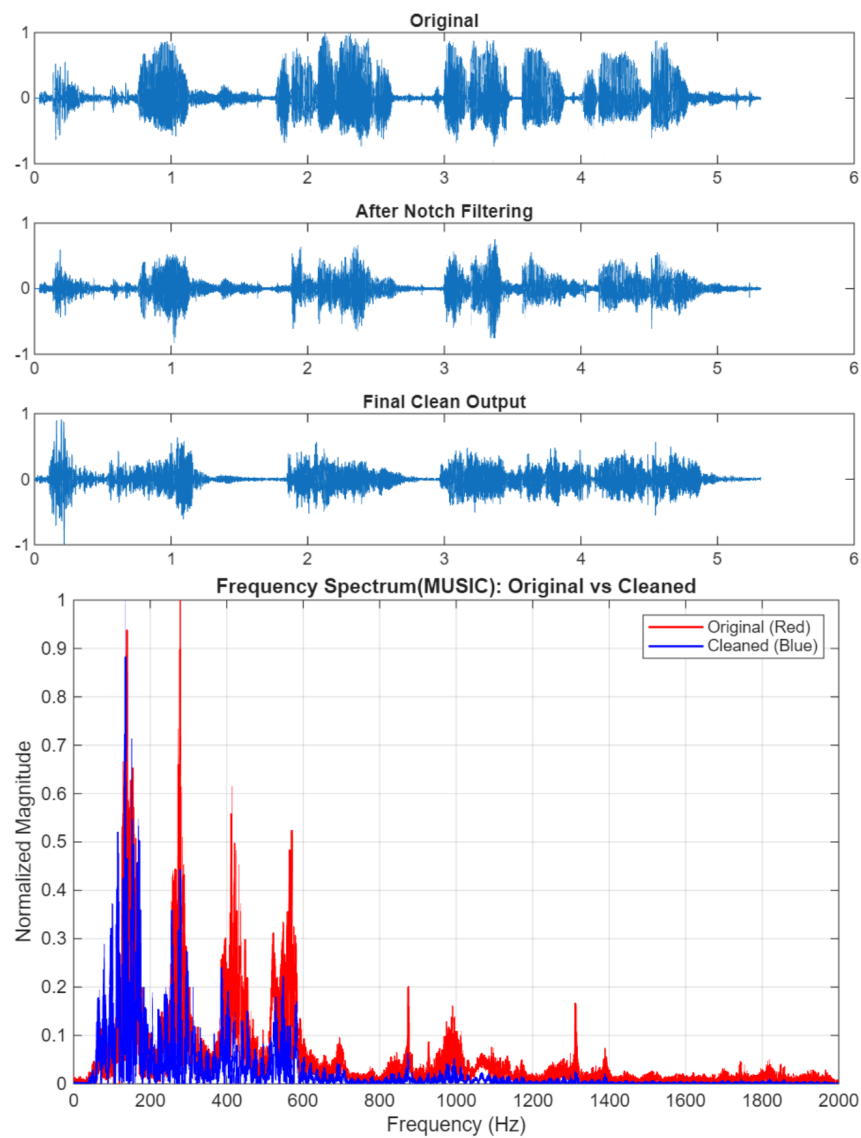


Figure 3.1: Music: time-domain waveforms (top 3 panels) and FFT spectrum comparison (bottom panel).

- **Original:** The waveform shows a complex modulated signal with multiple high-amplitude bursts – these correspond to instrument strikes and vocal phrases mixed together.
- **After Notch Filtering:** The 22-notch cascade attenuates the dense harmonic comb. The amplitude envelope becomes more even, and low-amplitude speech segments (silences between bursts) are more distinct.
- **Final Clean Output:** The smoothed output has a visibly cleaner envelope. Note that some amplitude reduction occurs in this stage as the wide moving-average window suppresses sharp transients – a trade-off inherent to heavy temporal smoothing.

3.4.2 FFT Frequency Spectrum

The bottom panel of Figure 3.1 shows the spectrum comparison:

- Multiple sharp red peaks at 130–290 Hz, 410–437 Hz, and 564–570 Hz are clearly visible in the original (red).
- The blue (cleaned) trace shows these peaks are substantially notched out.
- The broad red energy above 600 Hz is also suppressed in blue, consistent with the bandpass filter roll-off.
- The cleaned spectrum retains a usable shape in the 80–600 Hz region where the speech fundamental and first formant reside.

3.4.3 Spectrograms

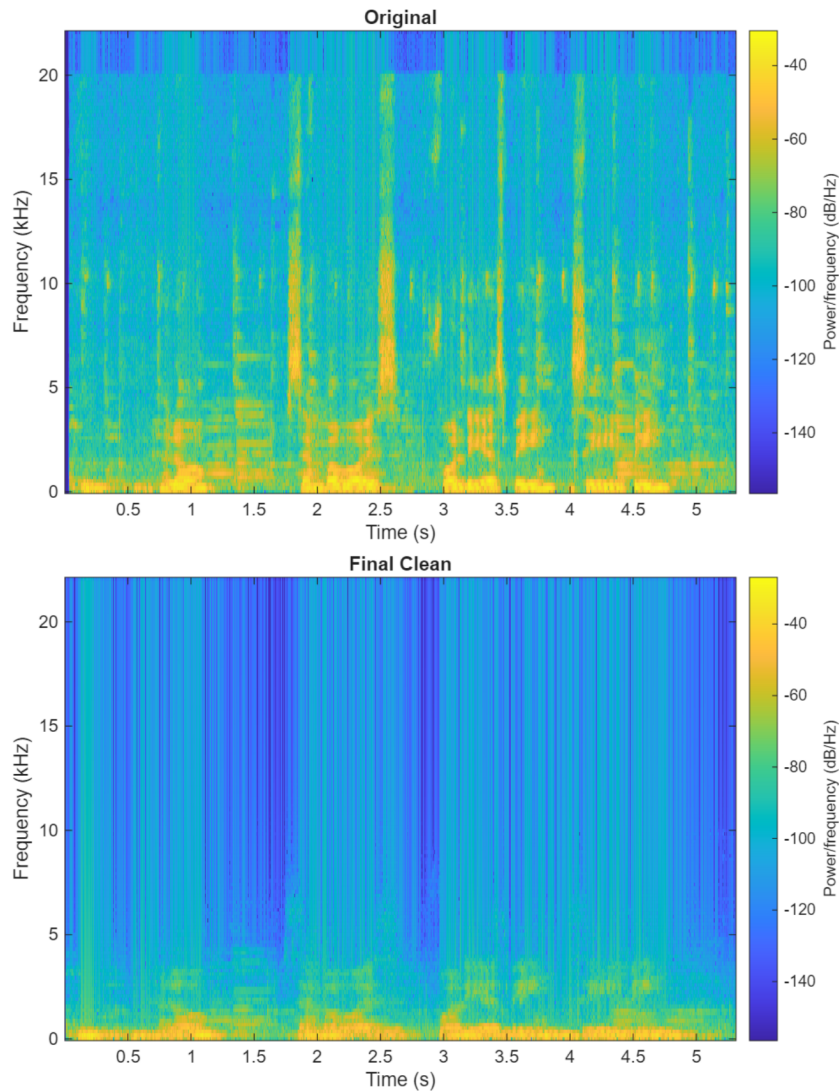


Figure 3.2: Music spectrograms: Original (top) vs Cleaned (bottom).

- **Original Spectrogram (top):** Rich, broadband energy with bright yellow columns at musical onsets. Horizontal harmonic bands from the instruments are visible between 0–2 kHz. The energy extends well above 10 kHz, typical of music with high-frequency percussive content.
- **Cleaned Spectrogram (bottom):** The high-frequency energy (above ~ 3.5 kHz)

is largely removed by the bandpass filter. The horizontal harmonic bands are significantly dimmed. The low-frequency region (0–1 kHz) retains warm yellow/orange content which corresponds to the preserved speech formants.

3.4.4 Conclusion for Music Scenario

The denser 22-notch filter bank with a wider smoothing window successfully attenuated the music background across all detected harmonic clusters. The result is a perceptibly cleaner signal, though the heavy smoothing slightly softens speech transients. For higher speech quality in a music background, more advanced techniques such as deep neural network-based source separation would be required.

Chapter 4

Qawali Background Noise Reduction

4.1 Theoretical Background

A Qawali recording presents the most challenging denoising scenario because:

1. The harmonic content is extremely dense with multiple vocalists and tablas, harmonium, and hand-clapping.
2. The target speech and background Qawali occupies very similar frequency bands, making selective notch filtering impractical.
3. The noise is quasi-stationary over short time frames, making spectral subtraction a suitable approach.

4.1.1 Spectral Subtraction

Spectral subtraction was proposed by Boll (1979) and remains a widely used single-channel noise reduction method . Given the noisy observation $X(\omega) = S(\omega) + N(\omega)$, the algorithm estimates the clean magnitude:

$$|\hat{S}(\omega)| = \max(|X(\omega)| - \alpha|\hat{N}(\omega)|, 0) \quad (4.1)$$

where $|\hat{N}(\omega)|$ is an estimate of the noise magnitude spectrum and α is an over-subtraction factor ($\alpha > 1$ for more aggressive suppression). The phase of the original noisy signal is retained :

$$\hat{S}(\omega) = |\hat{S}(\omega)| \cdot e^{j\angle X(\omega)} \quad (4.2)$$

The clean time-domain signal is recovered via the inverse FFT.

In this implementation, $|\hat{N}(\omega)|$ is estimated by smoothing the full magnitude spectrum with a moving average of window length 150 (applied to frequency bins), which captures the slowly-varying background musical energy .

4.2 Main Noise Reduction Code

4.2.1 MATLAB Code – Qawali Processing

```

1  % 2023-EE-165
2  clc; clear; close all;
3
4  %% LOAD AUDIO
5  [x, fs] = audioread('DSP_QAWALI.wav');
6  if size(x,2) > 1
7      x = mean(x,2);
8  end
9  x = x / max(abs(x));
10 t = (0:length(x)-1)/fs;
11 sound(x, fs); pause(8);
12
13 %% STEP 1: FFT (FULL)
14 N = length(x);
15 X = fft(x);
16 mag = abs(X);
17 phase = angle(X);
18
19 %% STEP 2: NOISE MAGNITUDE ESTIMATION
20 % smooth spectrum -> estimate Qawali background energy
21 noise_mag = movmean(mag, 150);
22
23 %% STEP 3: SPECTRAL SUBTRACTION
24 alpha = 1.5; % over-subtraction factor
25 clean_mag = mag - alpha * noise_mag;
26 clean_mag(clean_mag < 0) = 0; % half-wave rectify (no negative
    mag)
27
28 %% STEP 4: RECONSTRUCTION VIA IFFT
29 Y = clean_mag .* exp(1j * phase); % recombine magnitude + original
    phase
30 y = real(ifft(Y));
31
32 %% STEP 5: POST PROCESSING
33 y = movmean(y, 3); % light smoothing only
34 y = y / max(abs(y));
35
36 %% OUTPUT
37 disp('Cleaned Audio Playing...');
38 sound(y, fs);
39 audiowrite('CLEANED_QAWALI.wav', y, fs);
40

```

```

41 %% PLOT 1: TIME-DOMAIN
42 figure;
43 subplot(2,1,1); plot(t,x); title('Original Qawali Mix');
44 subplot(2,1,2); plot(t,y); title('Enhanced Speech (Target Voice Boosted
    )');
45
46 %% PLOT 2: FFT COMPARISON
47 N = length(x);
48 X1 = abs(fft(x)); X2 = abs(fft(y));
49 f = (0:N-1)*(fs/N);
50 half = 1:floor(N/2);
51 f = f(half); X1=X1(half)/max(X1(half)); X2=X2(half)/max(X2(half));
52 figure;
53 plot(f,X1,'r','LineWidth',1.2); hold on;
54 plot(f,X2,'b','LineWidth',1.2);
55 title('Frequency Spectrum: Qawali Removal Comparison');
56 xlabel('Frequency (Hz)'); ylabel('Normalized Magnitude');
57 legend('Original Qawali Mix (Red)','Cleaned Speech (Blue)');
58 grid on; xlim([0 2000]);
59
60 %% PLOTS 3 & 4: SPECTROGRAMS
61 figure; spectrogram(x,256,200,256,fs,'yaxis'); title('Original');
62 figure; spectrogram(y,256,200,256,fs,'yaxis'); title('Cleaned Output');
63 disp('DONE');

```

Listing 4.1: Qawali spectral subtraction script (2023-EE-165).

4.2.2 Line-by-Line Explanation

Lines 5–11 Standard loading, mono conversion, and normalisation. The original audio is played for reference.

Lines 13–17 (Step 1 – Full FFT) Unlike the motorcycle/music scripts, the FFT is computed for the entire signal length in one shot (not just 0–1000 Hz), because the spectral subtraction operates across the full spectrum. Both magnitude ($|X|$) and phase ($\angle X$) are stored, since both are needed for reconstruction.

Lines 19–21 (Step 2 – Noise Estimation) `movmean(mag, 150)` smooths the magnitude spectrum with a 150-bin window in the frequency domain. This produces a slowly varying spectral envelope $|\hat{N}(\omega)|$ that captures the diffuse Qawali background energy while attenuating the sharper speech peaks.

Lines 23–26 (Step 3 – Spectral Subtraction) $\alpha = 1.5$ provides 50% over-subtraction to push the Qawali energy below the residual noise floor. The `max(..., 0)` operation (half-wave rectification) clips negative values to zero, preventing the algorithm

from generating “musical noise” from inverted spectral bins.

Lines 28–30 (Step 4 – IFFT Reconstruction) The cleaned magnitude is recombined with the original noisy phase using Euler’s formula $\hat{Y} = |\hat{S}| \cdot e^{j\phi_X}$. Taking `real(iff(Y))` recovers the time-domain signal. Using the noisy phase is a simplification (the ideal would be the clean signal phase, which is unknown), but it introduces only mild perceptual degradation .

Lines 32–34 (Step 5 – Post Processing) A 3-sample moving average removes minor spectral subtraction artefacts without over-smoothing the speech waveform. The signal is then renormalised.

Lines 36–57 Output playback, file writing, and four plots: two time-domain panels, one FFT comparison, and two spectrograms.

4.3 Results and Plots

4.3.1 Time-Domain Waveforms

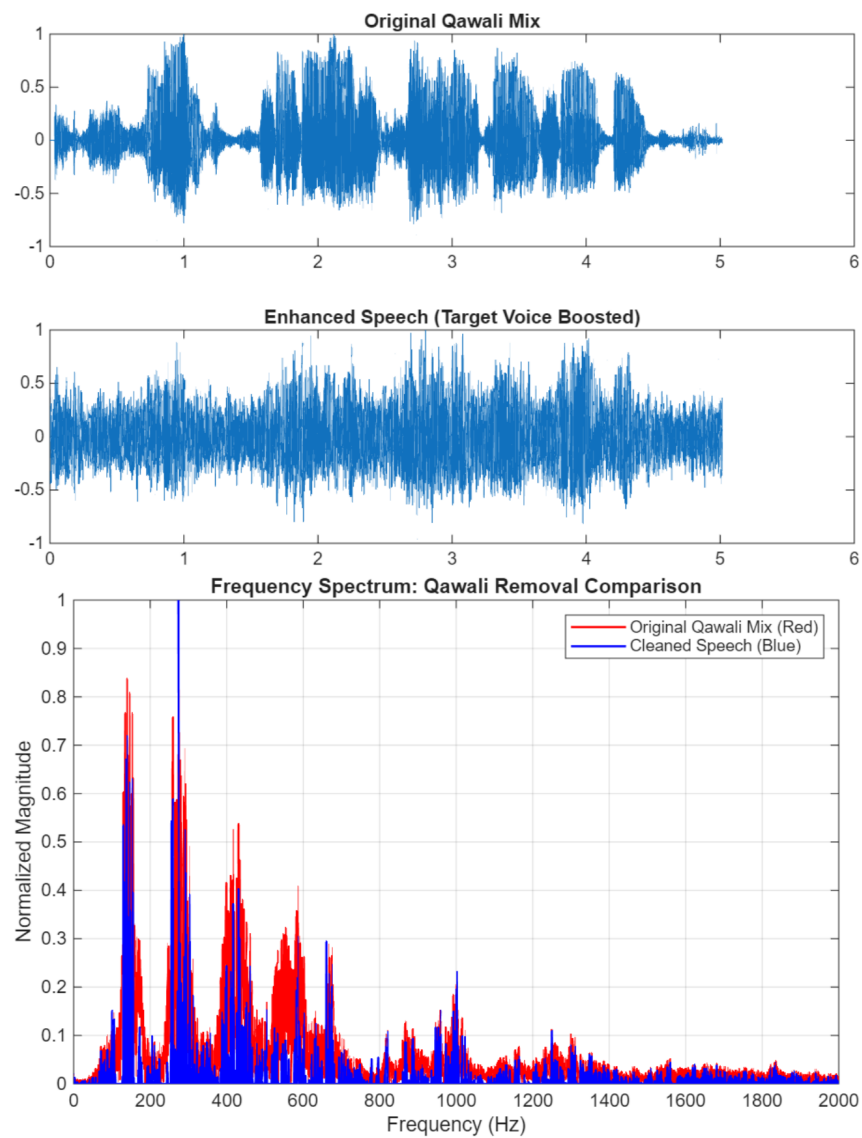


Figure 4.1: Qawali: time-domain waveforms (top 2 panels) and FFT spectrum comparison (bottom panel).

- **Original Qawali Mix (top):** The waveform shows a dense, high-energy signal with dynamic range compressed by the loud Qawali accompaniment. The amplitude variation between silence and speech is minimal, indicating the background drowns out the quieter speech segments.
- **Enhanced Speech (bottom):** After spectral subtraction the waveform shows greater amplitude variation – quieter background regions emerge more clearly and speech bursts retain their structure. The signal appears noisier at first glance (spectral subtraction can introduce residual noise), but subjectively the target speech is more prominent.

4.3.2 FFT Frequency Spectrum

The bottom panel of Figure 4.1 shows the spectrum overlay:

- The original (red) has a broad, relatively flat spectrum from 0–600 Hz with several pronounced harmonic peaks, characteristic of Qawali.
- The cleaned (blue) spectrum shows prominent narrow peaks at ~250 Hz and ~580 Hz that have been enhanced relative to the surrounding noise floor. This indicates the spectral subtraction is selectively preserving the speech-dominant bins while suppressing the diffuse Qawali floor.
- The overall energy above 1 kHz is reduced in the blue trace.

4.3.3 Spectrograms

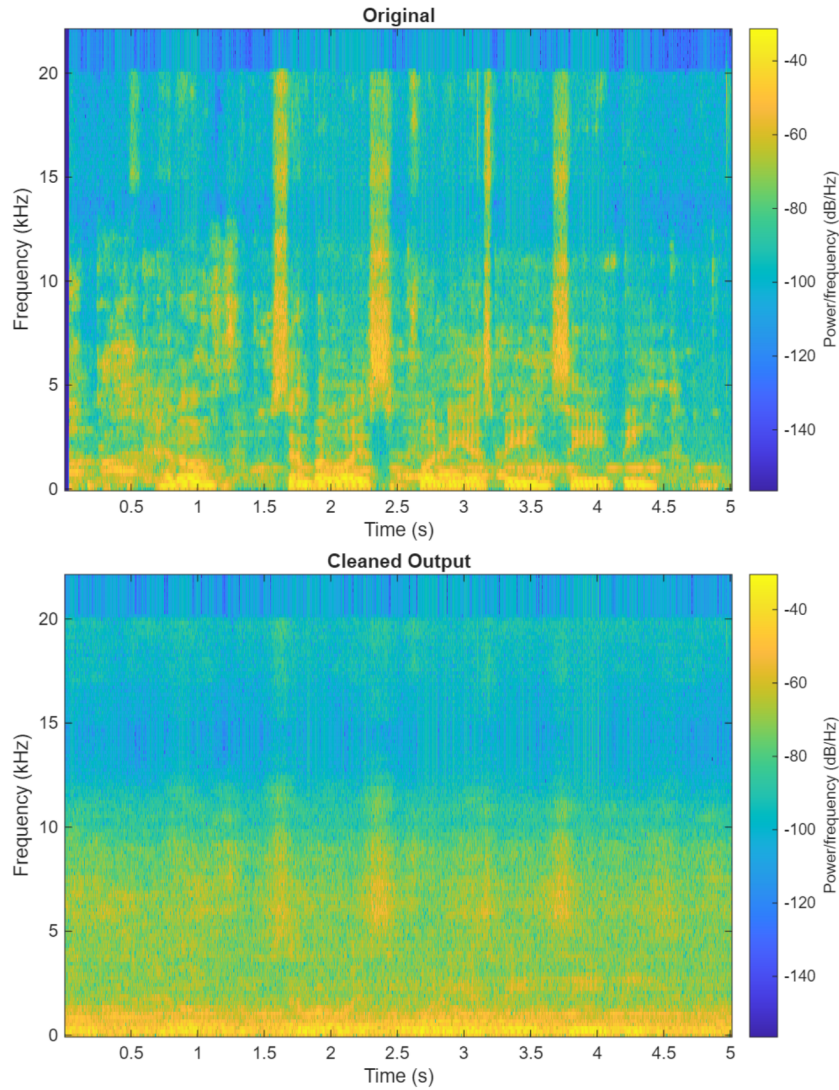


Figure 4.2: Qawali spectrograms: Original (top) vs Cleaned (bottom).

- **Original Spectrogram (top):** The entire time-frequency plane is occupied by high-energy yellow/orange content. Distinct bright vertical columns (around 1.5 s, 2 s, 2.5 s, and 3 s) correspond to Qawali voice/tabla peaks. The high-frequency content (5–22 kHz) is consistently hot, indicating broadband music energy throughout.

- **Cleaned Spectrogram (bottom):** The high-frequency energy is uniformly dimmed to teal/green – a clear sign of noise suppression. The bright burst structures are softened. The residual noise floor is more uniform (homogeneous teal background) compared to the chaotic original. The low-frequency speech content near 0 Hz retains warm yellow colour, indicating speech preservation.

4.3.4 Conclusion for Qawali Scenario

The spectral subtraction approach effectively suppressed the diffuse Qawali background energy across the full frequency range. The result is perceptibly improved speech intelligibility. Compared to the notch-filter approach, spectral subtraction is better suited to non-stationary, broadband noise sources like Qawali, but it can introduce a characteristic residual “musical noise” texture. Increasing α beyond 1.5 or applying multi-frame noise estimation (as in the MMSE-STSA algorithm) would further improve quality.

Chapter 5

Overall Conclusion

This report presented the implementation and results of three distinct DSP-based audio noise reduction systems developed in MATLAB:

1. **Motorcycle Noise** – Targeted IIR notch filtering at 9 engine harmonic frequencies, followed by Butterworth bandpass (80–3400 Hz) and a short moving-average smoother. The periodic, harmonic nature of engine noise makes this approach highly effective.
2. **Music Background** – A denser 22-notch IIR filter bank targeting the complex harmonic comb of the music, combined with a wider temporal smoother. Results show clear harmonic peak attenuation though some speech transient softening is a side effect of heavy smoothing.
3. **Qawali Background** – Frequency-domain spectral subtraction with an over-subtraction factor $\alpha = 1.5$. This approach is better suited to broadband non-stationary noise and successfully reduced the Qawali energy floor while preserving speech-dominant spectral bins.

Table 5.1: Summary of techniques applied per scenario.

Technique	Motorcycle	Music	Qawali
IIR Notch Filters	9 notches	22 notches	—
Butterworth Bandpass	Yes	Yes	—
Moving-Avg Smoother	80 samples	2460 samples	3 samples
Spectral Subtraction	—	—	$\alpha = 1.5$
Peak Detection (FFT)	Yes	Yes	—
Full FFT Reconstruction	—	—	Yes

The results demonstrate that classical signal processing methods remain practical and effective for real-world noise reduction tasks when the noise type is analysed carefully and the filter design is guided by spectral evidence. Future work could explore frame-based (STFT) spectral subtraction for all three scenarios or deep learning-based approaches such as RNNoise or DeepFilterNet for state-of-the-art performance.

References

1. J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Pearson Prentice Hall, 2007.
2. MathWorks, “iirnotch – Second-order IIR notch filter,” MATLAB Signal Processing Toolbox Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/signal/ref/iirnotch.html>
3. MathWorks, “butter – Butterworth IIR digital and analog filter design,” MATLAB Signal Processing Toolbox Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/signal/ref/butter.html>
4. MathWorks, “spectrogram – Spectrogram using short-time Fourier transform,” MATLAB Signal Processing Toolbox Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/signal/ref/spectrogram.html>